TP04 - Docker and Big Data Deployment

University of Eloued | Master 2: IA & Data Science | 2026/2027

Module: Big Data Analytics | GitHub: yacinebell39

1. Objective

The objective of this practical work is to package a Python data-analysis application with Docker, manage three containers on a shared network, and define the application environment with Docker Compose.

2. Application and Docker image

The application, `app.py`, reads Safecast CSV measurements and calculates record counts, numerical statistics, unit distributions, and counts above configured thresholds. It processes at most 100,000 rows per execution. It also provides a dashboard, a JSON endpoint at `/api/stats`, and a `/health` endpoint.

The Dockerfile uses `python:3.11-slim`, sets `/app` as the working directory, copies the source, dependencies and sample CSV, installs dependencies, and starts `python app.py`. The image name used in the project is `soufi39/tp04-data-analysis:latest`.

The installation and image workflow uses these commands:

```
docker --version
docker run hello-world
docker login -u soufi39
docker build -t soufi39/tp04-data-analysis:latest .
docker push soufi39/tp04-data-analysis:latest
docker pull soufi39/tp04-data-analysis:latest
```

3. Containers, network and ports

The project defines a bridge network named `tp04-net` and three containers named `safecast-node-1`, `safecast-node-2` and `safecast-node-3`. Host ports 8081, 8082 and 8083 map to port 5000 inside the respective containers. `docker ps` lists their state and port mappings.

The three containers run independent copies of the application. A shared network allows communication, but does not automatically divide the dataset between them.

4. Docker Compose and results

`compose.yaml` defines the `data-analysis` service and the `tp04-analysis` container. The bind mount `./data:/app/data:ro` makes local data available in read-only mode. With `/app/data` present, the application performs its analysis and exits. `restart: "no"` prevents automatic restarts.

```
docker compose up --build
docker compose ps -a
docker compose down
```

A local Python check on 2 November 2026 processed 10,000 sample rows and the first 100,000 rows of the 5.36 GB file. The application flagged 208 and 3,154 readings respectively. Its thresholds are greater than 50 for `cpm` and greater than 0.3 for `usv` or `usv/h`. These are application rules. Global averages mix units and should not be interpreted as a physical radiation measurement.

Verification note: Docker CLI version 29.9.1 was available, but the daemon was unavailable during this report check. The existing terminal images are generated illustrations; they are not independent execution evidence. The results above come from local Python execution.

5. Required questions

What is Docker, and why is it useful in data-science projects?

Docker packages an application and its runtime dependencies into an image that can run as an isolated container. In data science, it simplifies setup, reduces dependency conflicts, and helps reproduce the same software environment on different machines. Reproducible results also require consistent data, configuration and dependency versions.

What is the difference between an image and a container?

An image is a read-only template containing application files, dependencies and startup instructions. A container is an instance of that image with its own process environment and writable layer. Several containers can be created from one image, as demonstrated by the three-container setup.

What do FROM, WORKDIR, COPY, RUN and CMD do?

Instruction	Purpose in this project
FROM	Selects the base image: `python:3.11-slim`.
WORKDIR	Sets the working directory to `/app`.
COPY	Copies `requirements.txt`, `app.py` and the sample CSV into the image.
RUN	Installs Python dependencies while building the image.
CMD	Sets `python app.py` as the default startup command.

`RUN` executes during image creation; `CMD` defines what runs when a container starts.

What problem does Docker Compose solve?

Docker Compose stores service configuration in one YAML file. It makes containers, networks and mounts easier to configure and manage with repeatable commands. This project has one Compose service; the same file can be extended with additional services.

How does docker run compare with docker compose up?

`docker run` creates and starts a container using command-line options. It is convenient for individual tests. `docker compose up` reads a YAML file and creates or starts the services defined there. Compose is convenient when an application has several related services or repeated configuration.

How could this Compose project include a database?

Add a PostgreSQL service under `services`, configure its database credentials, and attach a named volume for persistent storage. The Python application would connect using the database service name as its hostname. It would also need a database driver and code to write results. A database health check and `depends_on` with `condition: service_healthy` can coordinate startup readiness. Adding a database container alone does not enable database storage in the current application.

6. Conclusion

This TP illustrates how Docker packages a Python application and how networks, port mappings and read-only mounts support deployment. Docker Compose provides a simple, reusable definition of the execution environment.